

SLM Harness MCP: Specialist Small-Model Subroutines for Cost-Efficient Coding Agents

Runtime merge run: LuxenAI/slmharness + IshaanAyaan/slm-agents

June 13, 2026

1 Abstract

This report documents an end-to-end merge of the slmharness runtime with research subroutine definitions from slm-agents. The merged runtime exposes JSON repair, traceback localization, search-query generation, and search-hit ranking through the existing MCP orchestration layer. The implementation keeps deterministic logic first, schema-SLM fallback second, and frontier handoff as a stub when no paid API key is configured.

2 Motivation

Coding agents repeatedly perform narrow support work that does not always require a frontier model. The harness routes those tasks to cheap verifiable specialists and escalates only when necessary.

3 Background: why naive small-model swapping fails

The research repo reports that small models work only when the task and harness are co-designed. Phase 2 decomposes developer-agent work into eight schema-verified subroutines and measures parameter floors from 135M to 1.5B.

4 System design

The runtime remains the product repo. The research repo remains external and contributes schemas, subroutine definitions, and benchmark framing. New runtime executors implement the existing BaseExecutor contract and return ExecutorResult telemetry.

5 Repo merge architecture

Runtime path: `/slm-mcp-merged/runtime`. Research path: `/slm-mcp-merged/research`. The integration plan is in `docs/MERGE_PLAN.md`.

6 MCP runtime design

The existing MCP server remains the entrypoint. New additive tools are `slm_repair_json`, `slm_localize_traceback`, `slm_generate_search_queries`, and `slm_rank_search_hits`.

7 Executor design

Each executor records executor name, model id, local/remote status, latency, estimated cost, success/failure, verifier result, fallback path, and trace id metadata.

8 Model selection from HF parameter-floor collection

Initial mapping: JSON repair uses `ishaanranjan/slm-agent-json-repair-smollm2-360m`; trace-back localization uses `ishaanranjan/slm-agent-trace-localizer-qwen2-5-0-5b`; search-query generation uses `ishaanranjan/slm-agent-search-query-gen-qwen2-5-0-5b`; search-hit ranking uses `ishaanranjan/slm-agent-search-hit-ranker-qwen2-5-0-5b`.

9 Experimental setup

The actual Lambda GPU reported by `nvidia-smi` was NVIDIA A10 with 23028 MiB VRAM, not A40. Benchmarks were intentionally small and controlled. Existing orchestration/MCP tests were run first.

10 Results

Mode	Success	Schema	Verifier	p50 ms	p95 ms	Fallback
deterministic_only	0.750	0.750	0.750	0.289	0.295	0.250
slm_only	1.000	1.000	1.000	0.038	0.093	1.000
deterministic_slm_fallback	1.000	1.000	1.000	0.035	0.086	0.250
deterministic_slm_frontier_stub	1.000	1.000	1.000	0.038	0.092	0.250
mcp_full_agent_path	1.000	1.000	1.000	0.170	0.290	0.250

The deterministic-only mode fails the malformed JSON case because it refuses to apply schema-SLM repair. All fallback modes repair the malformed JSON and pass controlled schema checks. Costs are zero in this local benchmark because no paid API key or live remote inference endpoint was used.

11 Failure analysis

The observed benchmark failure is intentional: deterministic-only JSON repair fails on Python-literal malformed JSON. No frontier fallback was live-tested because no API key was required or consumed. GPU memory readings are zero because this benchmark used CPU-local deterministic/schema fallbacks rather than loading HF checkpoints.

12 Cost model

Runtime cost is tracked per executor. Local deterministic and schema fallback paths use zero marginal API cost. Future live HF/vLLM runners should fill estimated cost using measured tokens/sec and GPU hourly rate.

13 Bigger-model integration path

Bigger models can call the subroutines through MCP and receive structured schemas plus trace telemetry. The frontier handoff is interface-complete but stubbed in this run.

14 Limitations

HF checkpoints are mapped but not live-loaded in this benchmark. The benchmarks are controlled smoke benchmarks rather than a full Phase 2 reproduction. Read-span selection is intentionally excluded until the first four subroutines are stable.

15 Next steps

Add optional transformers/vLLM-backed runners, re-run the matrix with live checkpoint inference, add real-repo tasks from the research fixtures, and wire live frontier fallback when API keys are available.

16 Appendix: commands, configs, schemas

```
cd ~/slm-mcp-merged/runtime
. .venv/bin/activate
pytest -q tests/test_orchestration tests/test_mcp
ruff check src/openharness/orchestration tests/test_orchestration scripts/smoke_mcp_subroutines.py
MYPYPATH=src mypy --explicit-package-bases src/openharness/orchestration tests/test_orchestration.py
python scripts/smoke_mcp_subroutines.py
python scripts/benchmark_mcp_subroutines.py
```

This paper distinguishes research repo evidence from newly generated runtime results. Research evidence is used for model selection and motivation. The benchmark table is newly generated by this merged runtime run.